

Fundamental data structures

Stacks

Stacks





What is a stack

- It is an abstract data type where new nodes can be added and removed only at the top (similar to a stack of dishes).
- Last-in, First-out (LIFO)
- Bottom of stack indicated by NULL



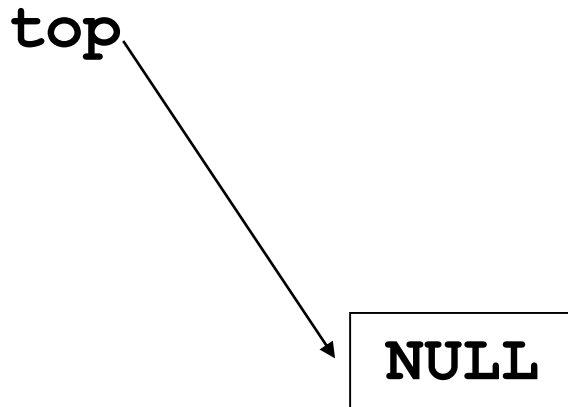
Operation possible on Stack

- Two major operations possible
- push
 - Adds a new node to the top of the stack
- pop
 - Removes a node from the top
 - Returns the popped value



List Stack Example

- **top** is a pointer to the top of the stack

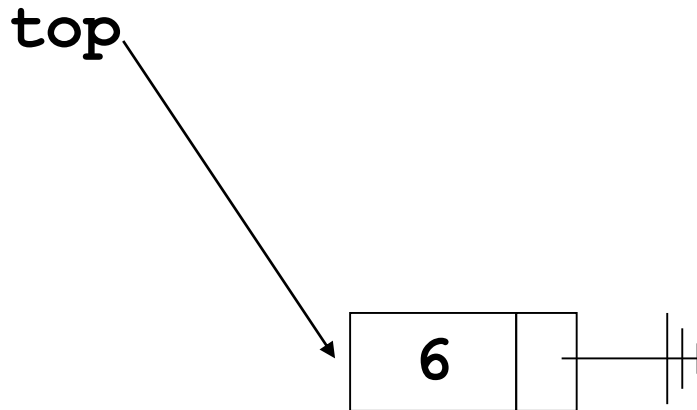


Empty Stack



List Stack Example

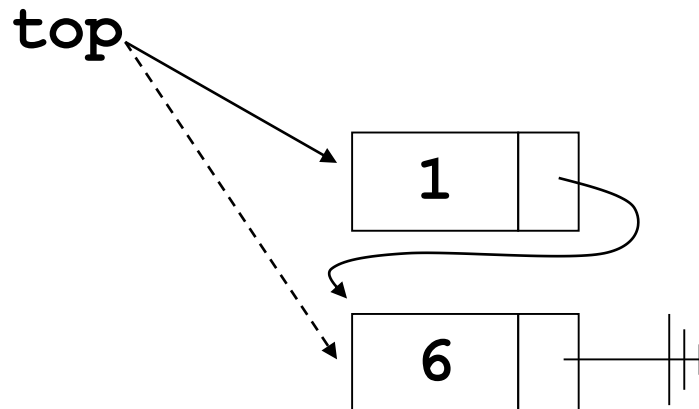
- The **push** operation inserts data element **6** at the top of the stack



push (6)



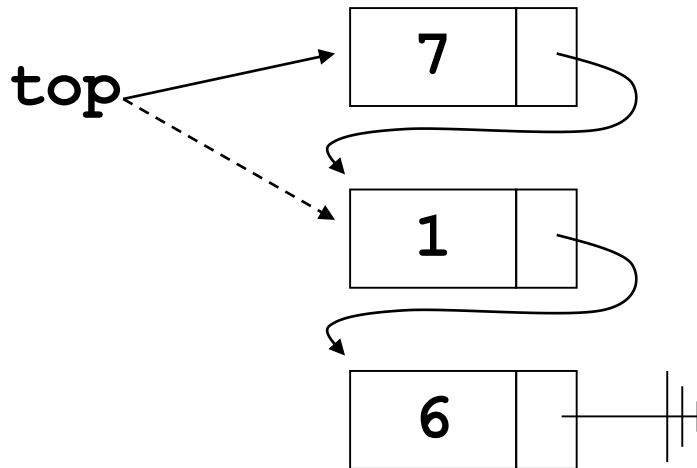
List Stack Example



push (1)

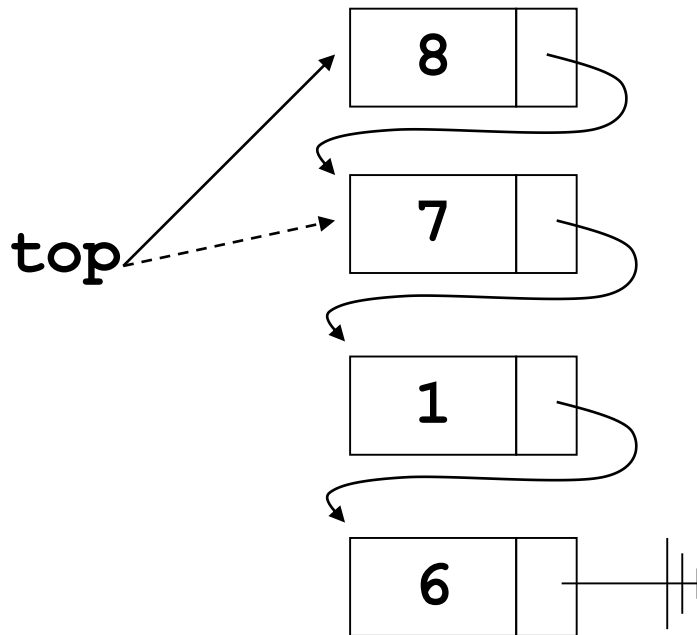
List Stack Example

- Observe how the stack grows and the value of the pointer top changes.



push (7)

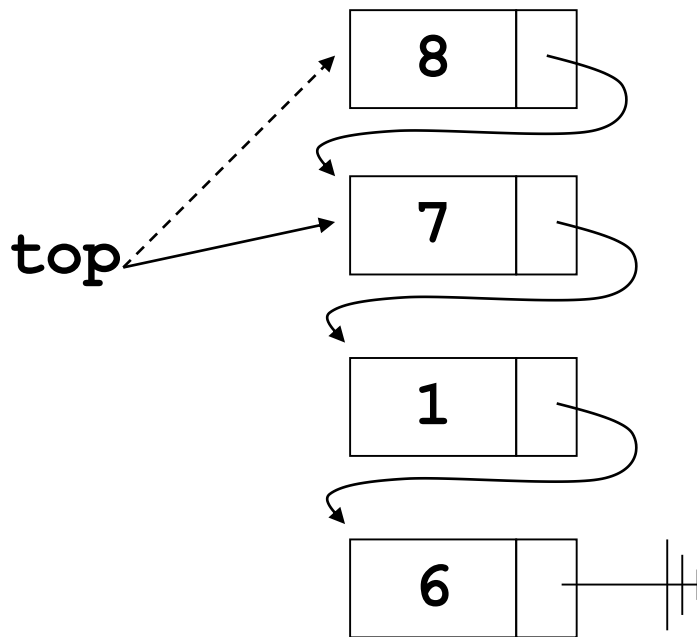
List Stack Example



push (8)

List Stack Example

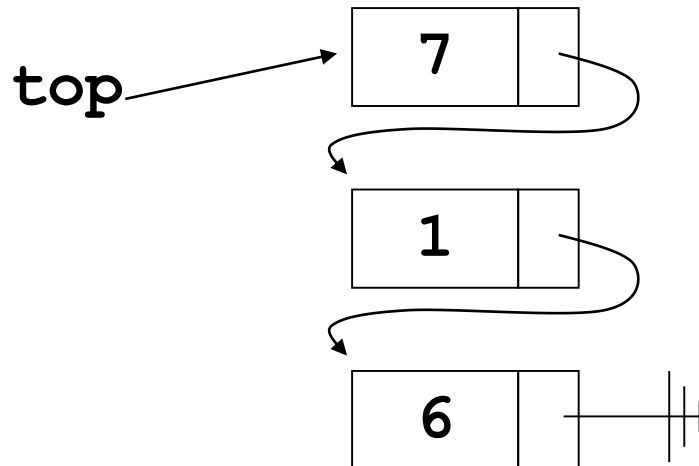
- The **pop** operation removes the topmost element of the stack



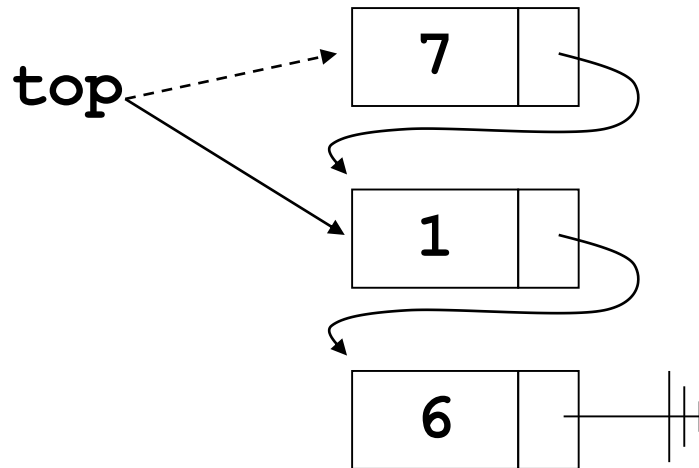
pop ()



List Stack Example



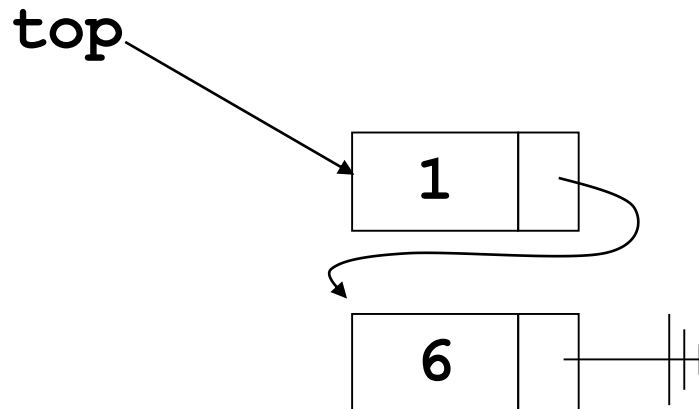
List Stack Example



pop ()



List Stack Example





Application of stacks

- Call stack (recursion)
- Searching Networks
 - Checking for balanced expressions
 - Recognizing palindromes
 - Evaluating algebraic expressions
- Traversing trees, etc



Stack in action

```
int factorial(int n) {  
    if (n == 0 || n == 1) // Base case  
        return 1;  
    else  
        return n * factorial(n - 1); // Recursive case  
}
```

- Suppose we call the function with $n = 5$ from `main()`
`int result = factorial(5);`



Checking for Balanced Braces

- A stack can be used to verify whether a program contains balanced braces

- An example of balanced braces

`abc { defg { ijk } { l { mn } } op } qr`

- An example of unbalanced braces

`abc { def } } { ghij { kl } m`

`abc { def } { ghij { kl } m`



Checking for Balanced Braces

- Requirements for balanced braces
 - Each time you encounter a “}”, it matches an already encountered “{”
 - When you reach the end of the string, you have matched each “{”.



Checking for Balanced Braces

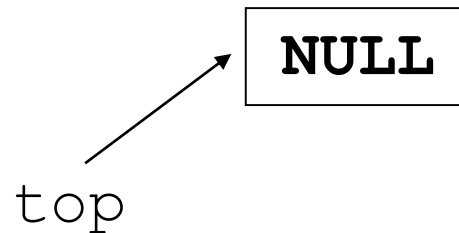
■ Input String ■ Stack

{ a { b } c }



■ Algorithm

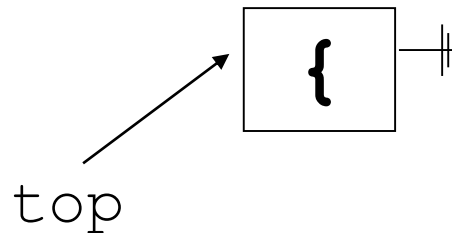
push ("{ ")



Checking for Balanced Braces

- Input String
- Stack
- Algorithm

{ a { b } c }

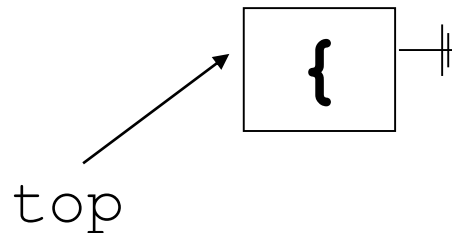




Checking for Balanced Braces

- Input String
- Stack
- Algorithm

{ a { b } c }



Checking for Balanced Braces

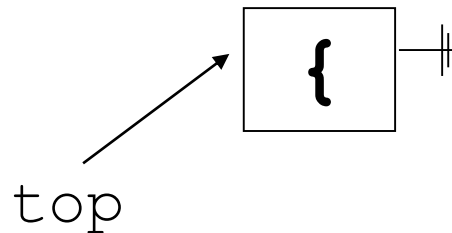
■ Input String ■ Stack

{ a { b } c }



■ Algorithm

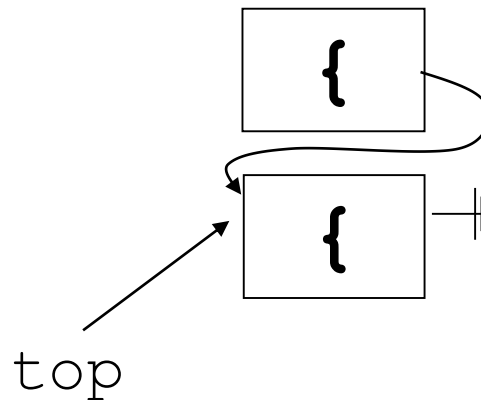
push ("{ ")



Checking for Balanced Braces

- Input String
- Stack
- Algorithm

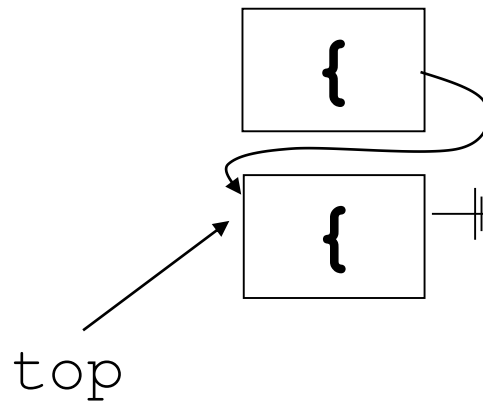
{ a { b } c }



Checking for Balanced Braces

- Input String
- Stack
- Algorithm

{ a { b } c }



Checking for Balanced Braces

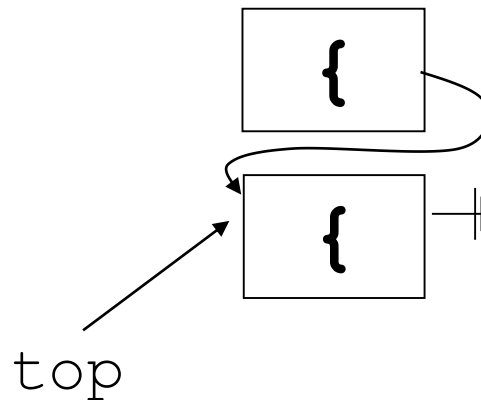
■ Input String ■ Stack

{ a { b } c }



■ Algorithm

pop ()

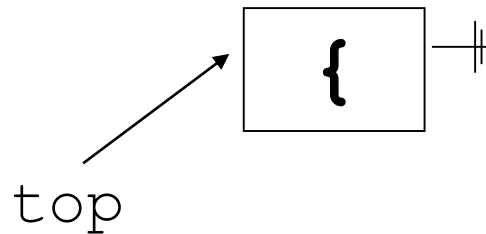




Checking for Balanced Braces

- Input String
- Stack
- Algorithm

{ a { b } c }

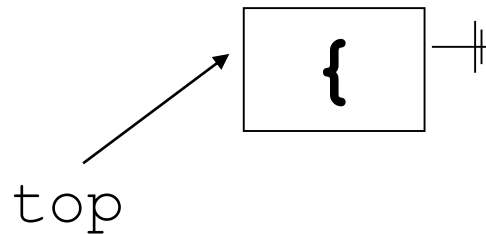




Checking for Balanced Braces

- Input String
- Stack
- Algorithm

{ a { b } c }



Checking for Balanced Braces

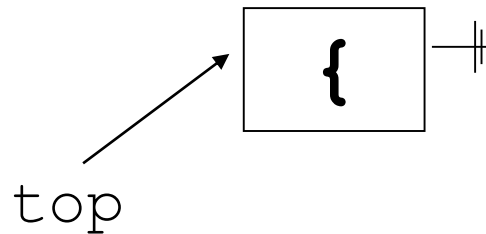
■ Input String ■ Stack

{ a { b } c }



■ Algorithm

pop ()

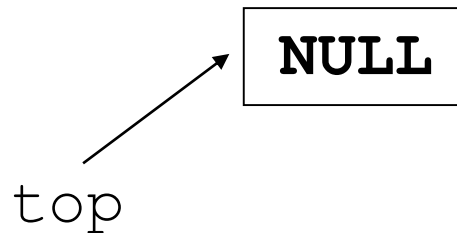




Checking for Balanced Braces

- Input String
- Stack

{ a { b } c }



- Algorithm

No input
and stack
empty.

**Braces
matched!**



Stack operations

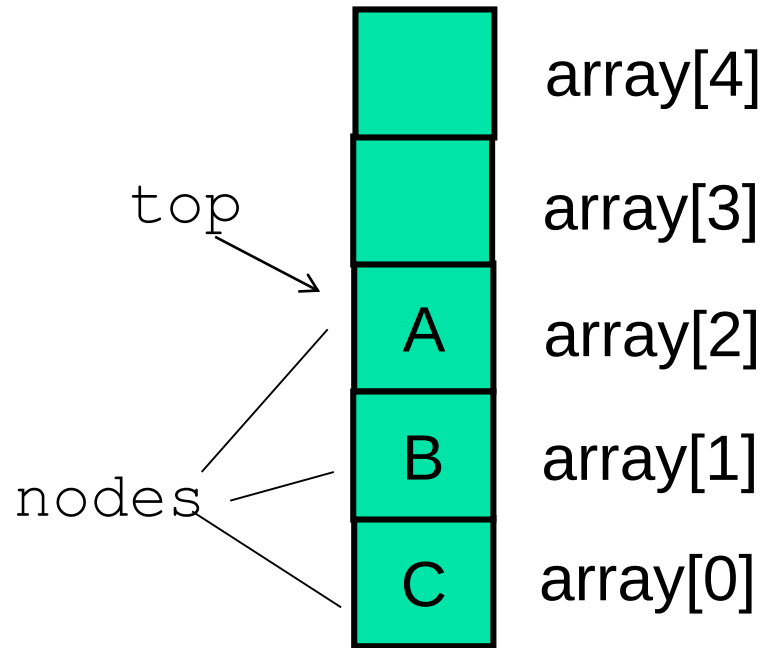
- Main operations
 - Push and Pop
- Other stack operations
 - initialize() - Initializes the stack
 - top_data() - Returns the topmost element of the stack
 - isEmpty() - Returns TRUE if stack is empty.
 - isFull() - Returns TRUE if stack is full.



Stack Implementation

- Stacks can be implemented using
 - Arrays
 - Linked List

Stack using Arrays



- A stack can be implemented using an array of pre-determined size.



Stack implementation

- Push element into stack
 - If stack is full, print error message
 - Else push element into stack
- Pop element from check
 - If stack is empty, print error message
 - Else pop element from stack



Stack implementation

1. Declare array
2. Function to initialize stack
3. Function to check if stack is empty
4. Function to check if stack is full
5. Push element into stack
6. Pop element from stack
7. Use the stack – push and pop from main()



Stack implementation

1. Declare array
 - We will use a structure with two members
 1. An integer array *items* to store stack elements
 2. A variable *top* to store index of top stack item



Stack implementation

1. Declare array
2. Function to initialize stack
 - Set top of stack to -1
 - `void initialize(Stack* stack)`



Stack implementation

1. Declare array
2. Function to initialize stack
3. Function *int isEmpty(Stack *stack)* to check if stack is empty
 - Returns TRUE (1) if top is set to -1
4. Function *int isFull(Stack *stack)* to check if stack is full
 - Returns TRUE if value of top is MAX_SIZE - 1



Stack implementation

1. Declare array
2. Function to initialize stack
3. Function to check if stack is empty
4. Function to check if stack is full
5. Push element into stack

void push(Stack stack, int item)*



Stack implementation

1. Declare array
2. Function to initialize stack
3. Function to check if stack is empty
4. Function to check if stack is full
5. Push element into stack
6. Pop element from stack

int pop(Stack stack)*



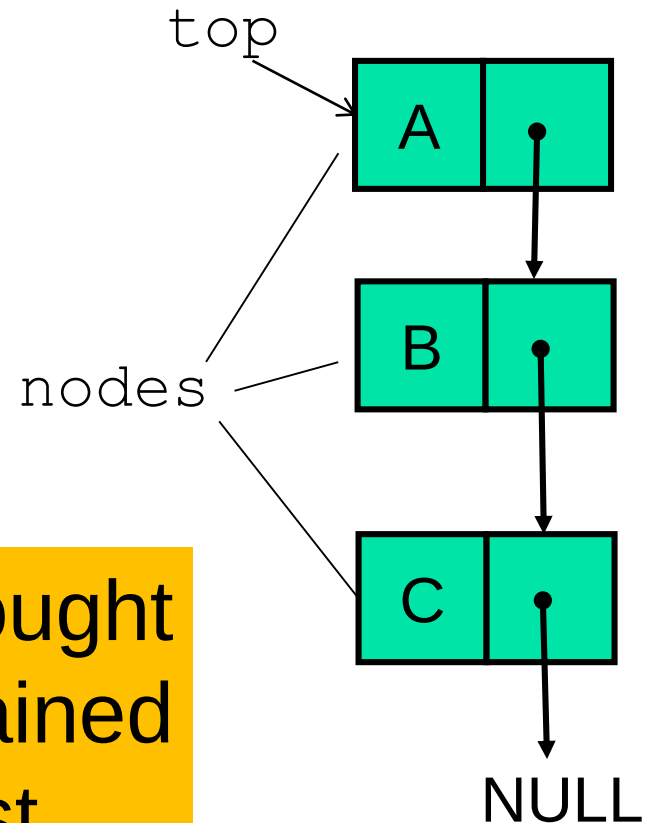
Stack implementation

1. Declare array
2. Function to initialize stack
3. Function to check if stack is empty
4. Function to check if stack is full
5. Push element into stack
6. Pop element from stack
7. Use the stack



```
int main() {  
    ■ Stack stack;  
    initialize(&stack);  
    push(&stack, 1);  
    push(&stack, 2);  
    push(&stack, 3);  
    printf("Popped element: %d\n", pop(&stack));  
    printf("Popped element: %d\n", pop(&stack));  
    printf("Popped element: %d\n", pop(&stack));  
    // Trying to pop from an empty stack  
    printf("Popped element: %d\n", pop(&stack));  
    return 0;  
}
```

Stack using Linked List



- A stack can be thought of as a constrained version of a linked list.



Stack implementation using Linked List

- Declare Node and Stack
- Function to initialize stack
- Function to check if stack is empty
- ~~Function to check if stack is full~~
- Push element into stack
- Pop element from stack
- Use the stack



Stack implementation using Linked List

- Declare Node and Stack

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node* next;
};
typedef struct node Node;
typedef struct {
    Node* top;
}Stack;
```



Stack implementation using Linked List

- Declare Node and Stack
- Function to initialize stack
void initializeStack(Stack stack)*
- Function to check if stack is empty
int isEmpty(Stack stack)*



Stack implementation using Linked List

- Declare Node and Stack
- Function to initialize stack
- Function to check if stack is empty
- Push element into stack

```
void push(Stack* stack, int data)
```



Stack implementation using Linked List

- Declare Node and Stack
- Function to initialize stack
- Function to check if stack is empty
- Push element into stack
- Pop element from stack



Stack implementation using Linked List

- Declare Node and Stack
- Function to initialize stack
- Function to check if stack is empty
- Push element into stack
- Pop element from stack
- Use the stack



```
int main() {  
    Stack myStack;  
    initializeStack(&myStack);  
    push(&myStack, 10);  
    push(&myStack, 20);  
    push(&myStack, 30);  
    printf("Popped element: %d\n", pop(&myStack));  
    printf("Popped element: %d\n", pop(&myStack));  
    printf("Popped element: %d\n", pop(&myStack));  
}
```